

AvatarGuru.ai

Project Technical Report

An AI-Powered Interactive Learning Platform

Date: 25 April 2026

1. Project Overview

AvatarGuru.ai is an AI-powered interactive educational platform designed to transform static PDF documents into an engaging, multi-modal learning experience. The platform leverages a 3D animated avatar, real-time AI tutoring, automatic speech synthesis, and an intelligent assessment engine to help students understand complex topics more effectively.

The system allows users to upload any PDF document — textbooks, lecture notes, research papers — and automatically processes it into structured, batch-wise explanations accompanied by curated reference links, educational images, and spoken audio narration delivered through a lip-synced 3D avatar.

1.1 Project Directory Structure

The project is organised as a full-stack application with the following top-level layout:

```
AvatarGuru.ai/
├── client/                # Frontend React/Vite source code
├── Documentation/         # Project reference and technical papers
├── public/                # Static frontend assets
├── tts_collections/       # Text-to-speech audio outputs/collections
├── uploads/              # User uploaded files/assets
├── venv/                  # Python virtual environment
├── node_modules/          # Node.js dependencies
└── [Configuration & Root Scripts...]
```

1.2 Core Features

- PDF upload and automated content extraction pipeline
- Batch-wise AI-generated summaries with educational image search
- 3D avatar with real-time audio-driven lip synchronisation
- RAG-based AI tutor chatbot with text and voice input
- AI-generated assessments: theoretical questions and MCQs
- Multilingual voice input with automatic translation to English
- Text-to-Speech (TTS) narration using Parler TTS and Groq Play.ai
- MongoDB + GridFS for document, audio, and metadata storage
- Qdrant vector database for semantic chunk retrieval

2. Frontend

The frontend is built with React, TypeScript, and Vite. It uses Tailwind CSS for styling, Three.js for 3D rendering, and ReactMarkdown with KaTeX for rich content display. The interface is structured around a three-tab layout: Upload, Learning Hub, and Assessment.

2.1 AvatarCanvas.tsx — 3D Avatar with Lip Sync (Study Page)

AvatarCanvas is the primary component responsible for rendering the animated 3D avatar and synchronising its mouth movements with audio playback. There are two versions of this component across the project — the Study page version (used in the Learning Hub) and a standalone version used on the home/hero page (Avatar.tsx). Both share the same core Three.js and Web Audio API architecture but differ in their model, camera setup, and audio-triggering mechanism.

The Study page AvatarCanvas accepts both a fileName and an audioUrl prop. The fileName allows the component to request TTS audio from the backend on demand, while audioUrl receives pre-generated audio URLs pushed from the parent when the AI tutor replies.

Key Responsibilities

- Initialises and manages a full Three.js scene lifecycle inside a React ref-based container
- Loads face9.glb GLTF model, centres and positions it, and traverses its mesh hierarchy to locate morph targets
- Detects the correct mouth morph target key from a prioritised candidate list: mouthOpen, JawOpen, viseme_aa, viseme_O, MouthOpen, mouth_open
- Maintains a persistent Web Audio API pipeline (AudioContext → MediaElementSource → AnalyserNode → destination)
- Drives the mouth morph target every animation frame based on real-time RMS amplitude computed from time-domain audio samples
- Accepts audioUrl as a prop — when the prop changes the component immediately stops any current audio and begins playing the new URL with lip sync
- Exposes a Play/Pause button that can also fetch audio from /api/learning-tts if no URL has been set yet

Three.js Scene Setup

On component mount a Three.js scene is created with a light-blue background (0xEEF5FD). A PerspectiveCamera with a 50-degree FOV is positioned at (0, 0.3, 1.3) looking toward (0, 0.2, 0) — a tight framing that keeps the avatar's face centred in the panel. An AmbientLight (0.8 intensity) provides base illumination and a DirectionalLight (1.5 intensity) from (0, 2, 2) adds depth shading. The WebGLRenderer is appended to the mountRef DOM element with antialiasing and device-pixel-ratio scaling for sharp rendering on high-DPI screens.

GLTF Model Loading & Morph Target Binding

The GLTFLoader fetches face9.glb from the /avatar/ public directory. Once loaded the model is centred using a THREE.Box3 bounding box and offset upward by 0.3 units. The component then calls model.traverse() over every node in the scene graph. For each mesh that has a morphTargetDictionary the component checks each candidate key in order. The first matching key is stored: the mesh itself in faceMeshRef and its integer morph target index in mouthIndexRef. frustumCulled is set to false on the bound mesh so Three.js never skips rendering it even when it partially leaves the camera frustum.

Web Audio API Pipeline

The Web Audio API pipeline is the heart of the lip sync system. It is constructed once — either lazily on first audio playback (Study page version) or eagerly on mount (Avatar.tsx version). The pipeline consists of three nodes connected in series:

- **AudioContext** — the root audio processing graph. One instance is created and reused for all subsequent audio. On browsers that auto-suspend AudioContext, `resume()` is called before each playback.
- **MediaElementSource** — wraps an `HTMLAudioElement` so its decoded PCM output flows into the Web Audio graph rather than directly to the speakers.
- **AnalyserNode** (`fftSize = 1024`) — taps into the audio stream and makes time-domain sample data available via `getByteTimeDomainData()`. It is connected to the AudioContext destination so audio still plays through the speakers while also being analysed.

The connection sequence is: `MediaElementSource → AnalyserNode → AudioContext.destination`. Every audio sample passes through the analyser before reaching the speakers, ensuring zero latency offset between what is heard and what drives the mouth movement.

Lip Sync — Frame-by-Frame Amplitude Calculation

Inside the `requestAnimationFrame` loop, if all four refs are non-null, the following calculation runs every frame:

1. `analyser.getByteTimeDomainData(dataArray)` — Fills the `Uint8Array` with 1024 time-domain samples. Each sample is an 8-bit value in `[0, 255]` where 128 = silence.
2. Normalise each sample: $v = (\text{sample} - 128) / 128$ — maps the range `[0, 255] → [-1.0, +1.0]`.
3. Compute RMS: $\text{rms} = \sqrt{\sum(v^2) / 1024}$ — measures the energy/loudness of the audio frame. Silence gives $\text{rms} \approx 0$; loud speech gives $\text{rms} \approx 0.05\text{--}0.15$.
4. Scale to morph target range: `lipValue = clamp(rms × 10, 0, 1)` — multiplying by 10 amplifies small RMS values into a visible `[0, 1]` mouth-open range.
5. Apply to mesh: `morphTargetInfluences[mouthIndex] = lipValue` — Three.js blends the neutral and open mouth shapes proportionally.

This approach is an amplitude-based approximation rather than true phoneme-driven viseme animation. It produces a convincing talking effect for educational narration without requiring a phoneme recognition pipeline.

audioUrl Prop — Reactive Playback

A separate `useEffect` watches the `audioUrl` prop. When it changes to a non-empty value the component resumes the `AudioContext`, pauses any currently playing audio, creates a new `Audio` element with `crossOrigin=anonymous`, wires it through the existing pipeline, and calls `play()`. This design means the parent (`LearningTab` or `AssessmentTab`) only needs to set a state variable to make the avatar start speaking — there is no imperative API to call.

Play/Pause Button (Study Page Version)

The Study page `AvatarCanvas` adds a circular button overlaid at the bottom-centre of the panel. Its behaviour depends on current state:

- If `isPlaying` is true — pause the audio and set `isPlaying = false`
- If audio is loaded and paused — resume it and set `isPlaying = true`
- If `audioUrl` is set but no audio element exists — call `playAudio(audioUrl)` to re-create the pipeline
- If no URL is available — POST to `/api/learning-tts` with the `fileName` prop, receive a URL, store it, and begin playback

A loading spinner (Loader2 from lucide-react) is shown while the TTS fetch is in progress. The button background changes from indigo (idle/paused) to rose (playing) to provide clear visual feedback.

2.2 Avatar.tsx — Standalone Home Page Avatar

Avatar.tsx is a simplified version of AvatarCanvas used on the home/hero page of the application. Unlike the Study page version it has no Play/Pause button, no TTS fetch logic, and no fileName prop. Its sole purpose is to render the avatar and animate its mouth whenever an audioUrl is passed in from the parent.

Differences from AvatarCanvas (Study Page)

- Uses girl3.glb instead of face9.glb — a different avatar model with a wider 100-degree FOV camera positioned at (0, 3.5, 2.5) for a full upper-body framing
- Targets the face mesh by exact name (Object_4) and directly accesses the mouthOpen morph target key without a candidate list
- AudioContext and AnalyserNode are created eagerly on mount rather than lazily on first play
- No isPlaying, loading, or audioUrl state — the component is entirely prop-driven and stateless regarding audio control
- Includes a window resize handler that updates renderer size and camera aspect ratio
- Cleanup on unmount removes the resize listener, disposes the renderer, and closes the AudioContext

2.3 StudyPage — Learning Hub & Assessment (Study.tsx)

Study.tsx is the main page component that orchestrates the entire study experience. It contains three tab components — UploadTab, LearningTab, and AssessmentTab — rendered conditionally based on the active tab state. A persistent navigation sidebar on the left links between tabs.

UploadTab

The upload tab provides a drag-and-drop zone accepting PDF files up to 50 MB. On drop or file selection it calls apiService.uploadFile(), polls /api/processing-status/:filename every 3 seconds, and shows a progress bar during upload and a spinner while the backend processes the document. On completion, a green success banner appears with a button to navigate to the Learning Hub. The tab also clears relevant localStorage cache keys on each new upload so stale summaries are never shown.

LearningTab

The Learning Hub is a three-panel resizable layout — AI Avatar, AI Tutor chat, and AI Summary — separated by draggable dividers. Each panel width is tracked in sectionWidths state and updated via mouse events on the resize handles.

AI Avatar Panel

Hosts the AvatarCanvas component. The avatarAudioUrl state is set whenever the AI tutor replies with a TTS audio URL, causing the avatar to start speaking the reply with lip sync.

AI Tutor Panel (Chat)

A chat interface that supports both text and voice input. Text messages are sent to `/api/qa` with the selected file name as context. Voice is captured via `MediaRecorder`, sent as `audio/webm` to `/api/qa-voice` which returns the transcript, the answer text, and a base64-encoded WAV audio response. Chat history is persisted to `localStorage` so it survives page refreshes.

AI Summary Panel

Displays the explanation batches fetched from `/api/get_batches`. Each batch contains a summary text, educational images, reference links, and keywords. The user can navigate between batches using Previous/Next buttons or dot indicators. A progress bar shows how many batches have been reviewed. Clicking an image opens a full-screen zoom modal. A Play Audio button sets the `avatarAudioUrl` to the batch's pregenerated audio URL so the avatar narrates the batch.

AssessmentTab

The assessment tab supports two modes: Theoretical (open-ended text answer) and MCQ (multiple choice). Clicking a generate button calls `/api/assessment/generate` with a `type` parameter. For MCQ, the raw response text is parsed by a regex-based `parseMCQ` function that extracts the question, four options, correct answer, explanation, and hint. The user selects an option and immediately receives correct/incorrect feedback. For theoretical questions, the user types an answer which is submitted to `/api/assessment/submit`; the backend returns structured feedback and a numeric score (0–100) displayed with colour coding — green for ≥ 80 , yellow for ≥ 50 , red otherwise.

LaTeX & Markdown Rendering

All AI-generated text in the study page is rendered through `ReactMarkdown` with the `remarkMath`, `remarkGfm`, and `rehypeKatex` plugins. A `preprocessLaTeX` helper converts `\[ ... \]` and `\( ... \)` delimiters to `$$` and `$` respectively before passing content to the renderer, ensuring mathematical expressions display correctly.

3. Backend

The backend is a Python Flask application that exposes a REST API consumed by the React frontend. It integrates MongoDB (with GridFS for binary storage), Qdrant vector database, the Groq cloud API for LLM and STT/TTS, a local Parler TTS model for high-quality narration, and the Serper.dev image search API.

3.1 server.py — Flask Application Entry Point

server.py is the main Flask server. It initialises all external service connections, registers the QA blueprint, and defines all non-QA API endpoints.

Initialisation

- Loads environment variables from .env using python-dotenv
- Connects to MongoDB and initialises GridFS for audio storage
- Creates the uploads/ and tts_collections/ folders if absent
- Registers the qa_bp Blueprint from qa_routes.py
- Imports shared helpers from common_fn.py
- Exposes shared DB objects (GRIDFS, FILES_COLLECTION, MONGO_CLIENT) to Blueprints via app.config

Key Endpoints

POST /api/upload

Accepts a PDF file, saves it to the uploads/ folder, inserts or updates a metadata document in the files MongoDB collection (including a deterministic qdrantCollection name via filename_to_collection_name), and spawns two subprocesses: ingest1.py (vector ingestion into Qdrant) and summary1.py (summary/image/audio generation). Returns 202 Accepted immediately so the frontend can poll for status.

GET /api/processing-status/<filename>

Reads the processingStatus field on the file document in MongoDB. Returns processing while the subprocesses are running and completed once both are done.

GET /api/files

Returns an ordered list of all uploaded file names from MongoDB, newest first. Used by the AI Tutor panel to populate the file selector.

GET /api/get_batches

Fetches the explanationBatches array from the most recently uploaded file document and returns it as an ordered JSON array. Each batch includes batchNumber, summary, links, keywords, images, and audio_url. The helper _get_file_context resolves documents from the files collection or lesson_contents collection, supporting both user uploads and preprocessed lessons.

GET /api/get_text

Returns a single concatenated summary string built from all batches of the latest file. Retained for backward compatibility with older frontend calls.

GET /api/get_links

Aggregates all reference links across all batches of the latest file and returns them as a flat array.

GET /api/tts-audio/<audio_id>

Streams a WAV audio file directly from MongoDB GridFS by ObjectId. Used by the avatar to play pregenerated TTS narration.

GET/POST /api/assessment/generate

Generates a question from the document using the MMR pool system. The `_get_next_chunk` function serves one diverse chunk at a time from a pre-built pool of 10 maximally-diverse chunks (retrieved via Max Marginal Relevance search from Qdrant). When the pool is exhausted it is automatically refilled. The LLM prompt varies by type (theoretical or mcq). For MCQ the strict format enforces Question / A-D options / Correct Answer / Explanation / Hint.

POST /api/assessment/submit

Accepts the original question and the student's answer. Fetches the full PDF text (up to `ASSESSMENT_MAX_CHARS = 15,000` characters) as context using `_get_assessment_context`, builds a structured evaluation prompt, and calls the Groq LLM. Returns a score (0–100) and formatted feedback text broken into status, correct points, improvement points, and key concept sections.

MMR Chunk Pool

The MMR pool (Max Marginal Relevance) is a module-level in-memory cache that prevents the same document chunk from being used for consecutive assessment questions. It uses LangChain's `max_marginal_relevance_search` to retrieve `MMR_TOP_K = 10` diverse chunks from Qdrant, setting `lambda_mult = 0.5` to balance relevance and diversity. A `_mmr_used` set tracks which chunks have been served. When all chunks are consumed the pool is automatically rebuilt.

Hierarchical Page APIs

- **GET /api/available-lessons** — Returns all preprocessed lessons stored in the `available_lessons` collection, exposing `board`, `subject`, `lesson`, `file_id`, and `title` fields.
- **GET /api/your-uploads** — Returns user-uploaded files with `file_id`, `filename`, and `upload date`, sorted newest first.
- **GET /api/lesson-content/<file_id>** — Fetches lesson content (explanation text and images) by `file_id`, checking `files` collection first, then `lesson_contents`.

3.2 qa_routes.py — Q&A Blueprint

`qa_routes.py` is a Flask Blueprint that handles all Q&A-related endpoints. It uses LangChain's RetrievalQA chain with a custom GroqLLM wrapper and the shared retrieval module.

GroqLLM Wrapper

A minimal LangChain LLM subclass that delegates all calls to `groq_generate` from `common_fn.py`. This allows RetrievalQA — which expects a LangChain-compatible LLM — to use the Groq cloud API without a native integration.

Collection Name Resolution

The `_resolve_collection_name` helper looks up the Qdrant collection name in two stages before falling back to a derived name: (1) checks the `files` collection by `originalName` for user-uploaded PDFs, (2) checks `lesson_contents` by `title` for preprocessed lessons. This ensures both upload paths use the correct vector store.

QA Prompt Template

All Q&A responses are generated with a shared prompt that instructs the model to produce student-friendly bulleted explanations. Every bullet must start with a dash and single space. Bold is reserved for part names and specific terms. No summary or conclusion is added at the end, keeping responses concise and scannable.

POST /api/qa

Text-based question answering. Derives the Qdrant collection name from the provided fileName, retrieves the top-2 most similar chunks via the retriever, passes them as context to the RetrievalQA chain, and returns the cleaned answer.

POST /api/qa-voice

Voice-based question answering pipeline. Validates audio size (max 25 MB), transcribes via Groq Whisper STT, optionally translates non-English input to English using a local LibreTranslate instance, runs the same RAG chain as /api/qa, synthesises the answer as speech via Groq Play.ai TTS, and returns transcript, response text, and a base64-encoded WAV audio URL in one response.

POST /api/qa-tts

Standalone TTS conversion endpoint. Accepts answer text, generates audio via groq_tts, stores the WAV bytes in MongoDB GridFS, writes a local copy to tts_collections/, associates the audio metadata with the file document in MongoDB, and returns the GridFS ObjectId. Used when the frontend requests TTS for a text-chat reply.

3.3 preprocess.py — Preprocessed Lesson Pipeline

preprocess.py processes instructor-curated PDFs into lessons that appear under Available Lessons in the hierarchical page. Unlike user uploads which are handled by summary1.py, these lessons are configured via .env variables and stored in dedicated MongoDB collections (available_lessons and lesson_contents) rather than the files collection.

Configuration (.env Variables)

PREPROCESS_PDF_PATH	– Path to the source PDF
PREPROCESS_BOARD	– Education board (e.g. CBSE)
PREPROCESS_SUBJECT	– Subject name
PREPROCESS_LESSON	– Lesson identifier
PREPROCESS_TITLE	– Display title
PREPROCESS_START_PAGE	– First page to process (1-based)
PREPROCESS_END_PAGE	– Last page to process (1-based)

The Qdrant collection name is derived automatically as {pdf_stem}_{start_page}_{end_page}, e.g. hist_12_15 for pages 12–15 of Hist.pdf. This is stored in the lesson_contents document for retrieval during Q&A.

Pipeline Steps

6. Load PDF with PyPDFLoader and filter to the configured page range via load_and_chunk_pdf.
7. Ingest chunks into Qdrant (creates or updates the collection) via ingest_into_qdrant.
8. Run process_batch_pipeline — generates summaries, links, keywords, images, and TTS audio per batch.
9. Upsert lesson metadata into available_lessons and full content into lesson_contents.

Idempotency Checks

Before running the full pipeline, `is_already_processed` checks the `lesson_contents` collection. If all data exists (explanation, batches, images, audio) it exits immediately. If only audio is missing it runs an audio-only pass regenerating WAV files for existing batches, updating each batch document individually in MongoDB. This prevents accidental data duplication on reruns.

3.4 ingest1.py — Vector Ingestion into Qdrant

`ingest1.py` handles the ingestion of a user-uploaded PDF into the Qdrant vector database. It is launched as a subprocess by `server.py` immediately after a file is uploaded.

Process

10. Load the PDF using `load_and_chunk_pdf` from `common_fn` — applying the `START_PAGE` and `END_PAGE` range from `.env`.
11. Call `ingest_into_qdrant` from `common_fn` to embed and upsert all chunks into the Qdrant collection.
12. Write the confirmed `qdrantCollection` name back to the file document in MongoDB so `qa_routes.py` can resolve it later.

Note: The collection name is derived deterministically from the PDF filename using the shared `filename_to_collection_name` helper in `common_fn.py` (extension stripped, lowercased, spaces replaced with underscores). Example: `Physics_Chapter1.pdf` → `physics_chapter1`.

3.5 retrieval.py — Qdrant Retriever

`retrieval.py` initialises the embedding model and Qdrant client once at module import time and exposes a `get_relevant_docs` function used by all Q&A and assessment endpoints.

Embedding Model

Uses `HuggingFaceBgeEmbeddings` with the `BAAI/bge-base-en-v1.5` model. If a local model path is set in `LOCAL_EMBEDDING_MODEL_PATH` and the directory exists, it is used directly to avoid downloading. `normalize_embeddings` is set to `False` to match the vectors stored during ingestion.

`get_relevant_docs(query, collection_name)`

Wraps a Qdrant collection as a `LangChain` retriever using similarity search with `k=2`. Returning the top 2 chunks keeps context concise and within the LLM's effective reasoning window. The function is called by both `/api/qa` and `/api/qa-voice` to fetch relevant document passages before passing them to the `RetrievalQA` chain.

3.6 summary1.py — User Upload Summary Pipeline

`summary1.py` is the second subprocess spawned after a user uploads a PDF. It fetches the latest uploaded document from MongoDB and runs the shared `process_batch_pipeline` to generate summaries, images, and audio. It then pushes each batch document into the file's `explanationBatches` array and marks processing as completed.

Idempotency

Before running the full pipeline, `summary1.py` checks whether the document already has batches with summaries, images, and audio. If all three exist it simply marks the document as completed and exits. If summaries and images exist but audio is missing it runs `regenerate_audio_for_existing_batches`, which iterates over batches without audio, generates WAV bytes via Parler TTS, stores them in GridFS, and updates the individual batch document.

3.7 `image_finder.py` — Educational Image Search

`image_finder.py` fetches educational diagram images from the Serper.dev Google Image Search API. It is called by `process_batch_pipeline` in `common_fn.py` with the keywords generated for each batch.

Rate Limiting

The `get_educational_images` function is decorated with `@sleep_and_retry` and `@limits` from the `ratelimit` library, respecting the `CALLS` and `PERIOD` values set in `.env` to avoid hitting Serper API rate limits.

Search Query

Each keyword is appended with the suffix 'educational diagram' before being submitted to Serper. The `safe=active` and `gl=in` parameters filter out inappropriate content and bias results toward Indian educational material. Only the first image URL from each query result is retained.

3.8 `common_fn.py` — Shared Utilities

`common_fn.py` is the central utility module imported by all backend scripts. It provides Groq API key rotation, LLM/STT/TTS wrappers, the Parler TTS pipeline, prompt builders, the batch processing pipeline, and the PDF chunking and Qdrant ingestion helpers.

Groq API Key Rotation

Multiple Groq API keys are loaded from a comma-separated `GROQ_API_KEY` environment variable. `execute_with_retry` wraps any Groq API call: if a call fails (rate limit, auth error, etc.) the key index is rotated to the next key and the call is retried up to `len(keys)` times. This provides seamless failover across keys without interrupting the pipeline.

`groq_generate`

Calls the Groq chat completions endpoint with configurable model, temperature, and `max_tokens`. Supports JSON-mode responses (`response_format: json_object`) used for structured links/keywords extraction. Defaults to the `SUMMARY_MODEL` from `.env`.

`groq_stt`

Transcribes audio bytes using Groq's Whisper-based STT model (distil-whisper-large-v3-en by default). Returns a transcription object with text and detected language.

`groq_tts`

Synthesises speech from text using the Groq Play.ai TTS endpoint. Voice, model, and output format are configurable via `.env` or per-call parameters. Audio bytes are accumulated from the streaming response and returned as raw bytes.

Parler TTS — Chunked Local Pipeline

`generate_audio_for_text` uses a locally hosted Parler TTS Mini v1 model for high-quality educational narration. Key implementation details:

- The model is loaded lazily on first call and cached in module-level globals (`_tts_model`, `_tts_tokenizer`). On CUDA it is loaded in fp16 to reduce VRAM consumption on hardware such as the RTX 3050 4 GB.
- **Sentence-aware chunking** — `split_text_into_tts_chunks` divides input text into sentence-aware chunks targeting `TTS_CHUNK_TARGET_TOKENS` (120) with a hard ceiling of `TTS_CHUNK_MAX_TOKENS` (180). Sentences are never cut mid-way unless a single sentence exceeds the ceiling.
- **Per-chunk generation** — `_generate_chunk_audio` generates audio for one chunk at a time. GPU tensors are explicitly freed after each chunk and `torch.cuda.empty_cache()` is called to release VRAM.
- **Retry & resume** — each chunk is attempted twice before being skipped. If `temp_chunk_dir` is provided, successfully generated chunks are saved as individual WAV files so the pipeline can resume from the last successful chunk on failure.
- **Merge & normalise** — all chunk arrays are concatenated and normalised to `[-1, 1]` to prevent clipping artefacts at join points, then serialised to WAV bytes via `scipy.io.wavfile`.

`process_batch_pipeline`

The core shared pipeline function. It processes document chunks in groups of 10 (configurable via `batch_size`). For each batch it: (1) generates per-chunk summaries and merges them, (2) generates links and keywords in JSON mode, (3) fetches educational images via `image_finder`, (4) generates TTS audio via the chunked Parler pipeline and stores it in GridFS, and (5) assembles and returns a batch document dict. The caller is responsible for persisting the returned dicts to MongoDB.

`load_and_chunk_pdf`

Loads a PDF with `PyPDFLoader` and splits it into overlapping chunks with `RecursiveCharacterTextSplitter` (`chunk_size=500`, `chunk_overlap=50`). Accepts optional `start_page/end_page` parameters (1-based) for page-range filtering. Used by both `ingest1.py` and `preprocess.py`.

`ingest_into_qdrant`

Embeds and stores document chunks in a Qdrant collection. Creates the collection if it does not exist using `QdrantVectorStore.from_documents`; appends to an existing collection using `vectorstore.add_documents`. Uses `HuggingFaceEmbeddings` with the model name from the `EMBEDDING_MODEL` environment variable.

`build_prompt_summary` / `build_prompt_links`

Standardised prompt templates used across all summary generation. `build_prompt_summary` instructs the model to generate a 5–8 sentence student-friendly explanation without links or keywords. `build_prompt_links` requests exactly 3 trusted URLs and 4 key phrases returned as JSON.

3.9 installation.py — TTS Model Download Utility

`installation.py` is a one-time setup script that downloads and saves the Kokoro-82M text-to-speech model from Hugging Face to a local directory. It authenticates using a HuggingFace token, loads the model with `AutoModelForCausalLM` and the tokenizer with `AutoTokenizer`, then calls `save_pretrained` to persist both to the configured path. A `BitsAndBytesConfig` block for 4-bit

quantisation is included but commented out, allowing it to be enabled on VRAM-constrained hardware.

4. Technology Stack Summary

Layer	Technology	Purpose
Frontend	React + TypeScript + Vite	UI framework and build tool
Frontend	Three.js + GLTFLoader	3D avatar rendering
Frontend	Web Audio API	Real-time lip sync amplitude analysis
Frontend	ReactMarkdown + KaTeX	Math and markdown rendering
Frontend	Tailwind CSS	Utility-first styling
Backend	Python Flask	REST API server
Backend	LangChain	RAG chain and prompt management
Backend	Groq API	LLM (chat), STT (Whisper), TTS (Play.ai)
Backend	Parler TTS Mini v1	Local high-quality audio narration (chunked pipeline)
Backend	Qdrant	Vector database for semantic retrieval
Backend	MongoDB + GridFS	Document storage and audio file storage
Backend	BAAI/bge-base-en-v1.5	Text embedding model
Backend	Serper.dev API	Educational image search
Backend	LibreTranslate	Multilingual voice input translation

5. End-to-End Data Flow

5.1 Document Upload & Processing

13. User uploads a PDF via the Upload tab.
14. Frontend calls POST /api/upload.
15. server.py saves the file, inserts a MongoDB document with processingStatus: processing, and assigns a qdrantCollection name.
16. Two subprocesses are spawned: ingest1.py and summary1.py.
17. ingest1.py loads the PDF, chunks it, embeds it via BAAI/bge-base-en-v1.5, and stores vectors in Qdrant. It then writes the confirmed collection name back to MongoDB.
18. summary1.py generates batch summaries, fetches images, synthesises TTS audio via the chunked Parler pipeline, stores audio in GridFS, and updates MongoDB with explanationBatches.
19. Frontend polls /api/processing-status/:filename until status becomes completed.
20. User is directed to the Learning Hub.

5.2 Learning Hub Session

21. LearningTab fetches /api/get_batches and renders each batch with summary text, images, and reference links.
22. User clicks Play Audio — the batch audio_url is set as avatarAudioUrl; AvatarCanvas plays it with lip sync.
23. User types or speaks a question in the AI Tutor panel.
24. Voice input: audio → /api/qa-voice → STT → optional translation → RAG → TTS → base64 WAV → avatar plays it.
25. Text input: question → /api/qa → RAG → text answer displayed in chat; optional TTS via /api/qa-tts.

5.3 Assessment Session

26. User selects question type (Theoretical or MCQ).
27. Frontend calls /api/assessment/generate?type=...&fileName=...
28. server.py fetches next chunk from MMR pool (or refills pool from Qdrant) and generates a question.
29. For MCQ: frontend parses the structured text response and renders option buttons.
30. User submits answer to /api/assessment/submit.
31. LLM evaluates against the full PDF context, returns score + structured feedback.
32. Feedback is rendered with a colour-coded score badge.

6. Environment Configuration (.env)

The application is configured entirely through environment variables loaded from a .env file at startup.

```
# Groq API (comma-separated for key rotation)
GROQ_API_KEY=gsk_key1,gsk_key2,...

# Models
SUMMARY_MODEL=llama-3.1-8b-instant
STT_MODEL=distil-whisper-large-v3-en
TTS_MODEL=playai-tts
TTS_VOICE=autumn

# MongoDB
MONGODB_URL=mongodb://localhost:27017
MONGO_DB=avatarguru
GRIDFS_COLLECTION=tts_audio

# Qdrant
QDRANT_URL=http://localhost:6333

# Embedding
EMBEDDING_MODEL=BAAI/bge-base-en-v1.5
LOCAL_EMBEDDING_MODEL_PATH=C:/path/to/local/model

# Storage
UPLOAD_FOLDER=uploads
TTS_OUTPUT_FOLDER=tts_collections
COLLECTIONS_FOLDER=/collections

# Serper Image Search
SERPER_API_KEY=your_serper_key
SERPER_URL=https://google.serper.dev/images
CALLS=5
PERIOD=60

# Translation
TRANSLATE_MODEL=http://localhost:5001/translate

# Ingest page range (for user uploads)
START_PAGE=1
END_PAGE=50

# Preprocess lesson config
PREPROCESS_PDF_PATH=C:/path/to/lesson.pdf
PREPROCESS_BOARD=CBSE
PREPROCESS_SUBJECT=Physics
PREPROCESS_LESSON=Chapter 1
PREPROCESS_TITLE=Physical World
```

PREPROCESS_START_PAGE=1
PREPROCESS_END_PAGE=20

— *End of Report* —